

AUTOMOTIVE DIAGNOSTIC ECU

STM32 • CAN • ISO-TP • UDS

Thành viên

Bùi Tiến Dũng - 23020792

Bùi Xuân Thanh - 23020883

Nguyễn Đình Cường - 23020786

Nguyễn Trường Giang - 23020802

Nền tảng

STM32F103 • Embedded C

Communication

CAN 2.0 + ISO-TP

Diagnostics

UDS / ISO 14229

NỘI DUNG TRÌNH BÀY

02

01

Bối cảnh & mục tiêu

02

Kiến trúc hệ thống

03

CAN / ISO-TP / UDS

04

Application & dữ liệu ECU

05

Test & traceability

06

Đóng góp & kế hoạch

1.1. BỐI CẢNH & MỤC TIÊU

03

Bài toán

Xây dựng một ECU trên STM32 có khả năng:

- Truyền dữ liệu trạng thái động cơ qua mạng CAN.
- Tiếp nhận và xử lý các yêu cầu chẩn đoán UDS.
- Phản hồi dữ liệu chẩn đoán theo đúng yêu cầu từ Tester.

Tư duy V-Model

Yêu cầu → Thiết kế → Lập trình → Kiểm thử → Minh chứng → Hoàn thiện

- Mỗi yêu cầu của hệ thống đều được liên kết với thiết kế, mã nguồn và các bài kiểm thử tương ứng để đảm bảo có thể kiểm chứng được toàn bộ quá trình phát triển.

1.2. PHẠM VI DỰ ÁN

04

IN SCOPE — PHẠM VI THỰC HIỆN

Nền tảng & giao tiếp

- Firmware Embedded C trên STM32F103
- CAN Controller + CAN Transceiver
- Truyền thông CAN giữa ECU và Tester

Chẩn đoán UDS

- Truyền EngineStatus theo chu kỳ qua CAN
- UDS Diagnostic Server
- Hỗ trợ ISO-TP cơ bản cho payload > 7 byte
- DID Manager và DTC Manager
- Các dịch vụ UDS theo yêu cầu hệ thống

Kiểm thử & quản lý

- CAN Trace và UART Log
- Lưu Test Evidence
- Quản lý GitHub: Issue → Branch → Pull Request → Merge

OUT OF SCOPE — KHÔNG THỰC HIỆN

Tích hợp xe thực tế

- Không kết nối trực tiếp với xe thật
- Không triển khai OBD-II thực tế

Các thành phần cấp sản phẩm

- Không xây dựng Full AUTOSAR Stack
- Không yêu cầu ISO-TP Timing ở mức Production
- Không xây dựng Production Bootloader
- Không triển khai Security Access hoàn chỉnh

Lưu trữ & an toàn

- Không lưu DTC trên NVM lâu dài
- Không thực hiện chứng nhận Functional Safety
- Không thực hiện chứng nhận Cybersecurity

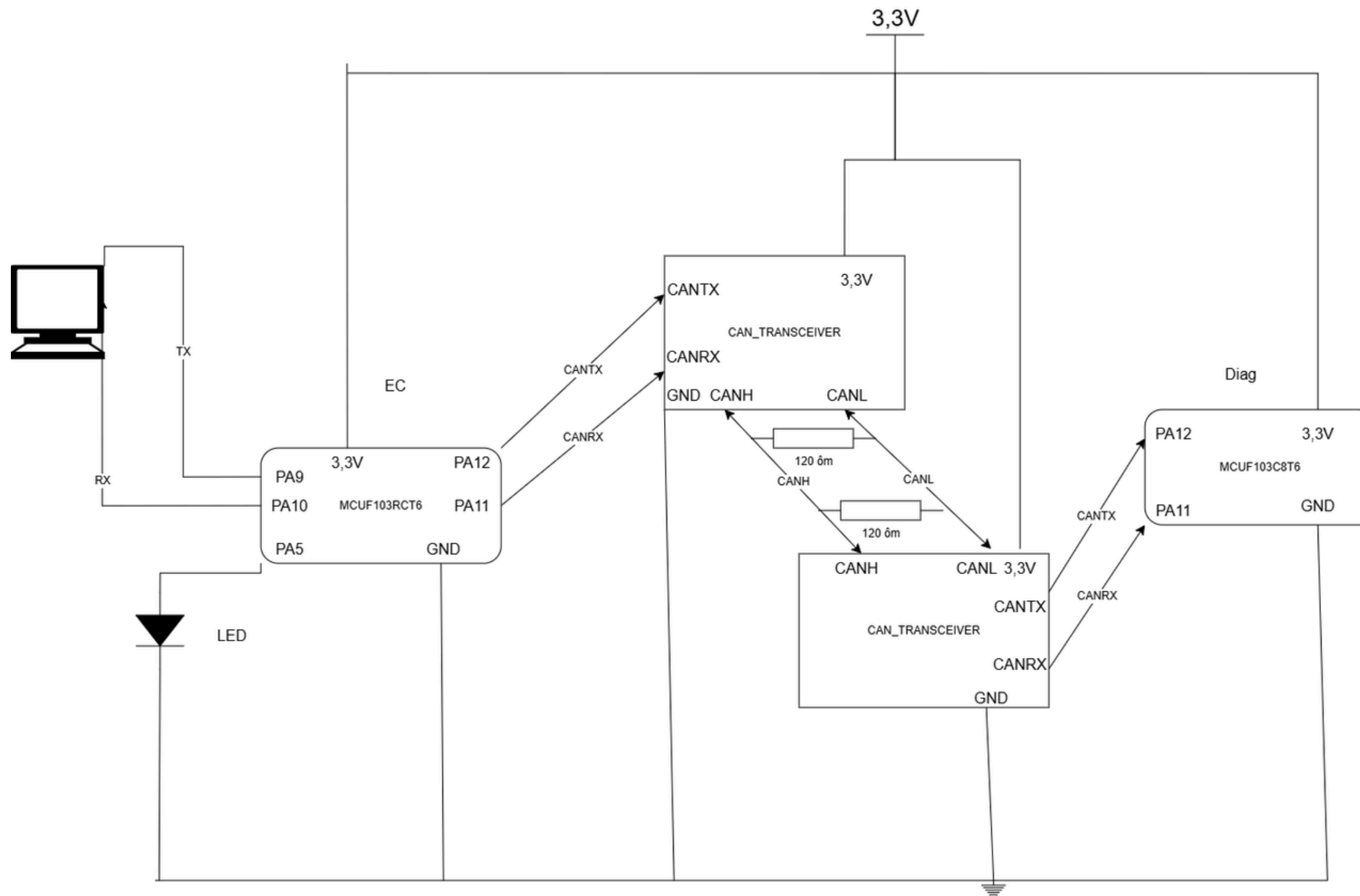
1.3. SYSTEM REQUIREMENTS

SYS-001	EngineStatus	ECU truyền EngineStatus qua CAN với ID 0x100, DLC 8, chu kỳ 1000 ms $\pm 10\%$, gồm VehicleSpeed, EngineRPM, EngineTemp và BatteryVoltage.
SYS-002	UDS communication	ECU hỗ trợ giao tiếp UDS trên CAN với các dịch vụ 0x10, 0x11, 0x19, 0x22 và 0x3E
SYS-003	VIN	ECU cung cấp VIN qua DID 0xF190. Request 22 F1 90 → Response 62 F1 90 <VIN>.
SYS-004	SW version	ECU cung cấp phiên bản phần mềm qua DID 0xF187. Request 22 F1 87 → Response 62 F1 87 <version>.
SYS-005	Dynamic data	ECU cung cấp dữ liệu động qua các DID 0x0101–0x0104: VehicleSpeed, EngineRPM, EngineTemp và BatteryVoltage.
SYS-006	DTC management	ECU hỗ trợ quản lý DTC, bao gồm các DTC mẫu 0x010001 – Engine Over Temperature và 0x010002 – Low Battery.
SYS-007	Negative response	ECU trả về Negative Response khi request không hợp lệ theo định dạng 7F <SID> <NRC>.
SYS-008	Real-hardware demo	Hệ thống được kiểm thử trên STM32 thực tế, sử dụng CAN Trace và UART Log làm bằng chứng.

2.1. HARDWARE ARCHITECTURE

06

Sơ đồ kiến trúc phần cứng hệ thống ECU



MCU

- Chạy firmware Embedded C và xử lý logic ứng dụng.
- Điều khiển CAN và giao tiếp với các module phần mềm.
- Kết nối với CAN Controller thông qua giao diện CAN tích hợp.

CAN controller

STM32 bxCAN + HAL_CAN

- Thực hiện truyền và nhận CAN Frame.
- Quản lý CAN ID, DLC và dữ liệu truyền nhận.
- Cung cấp giao diện giữa firmware và CAN Transceiver.

Transceiver + bus

Lớp giao tiếp vật lý

- Chuyển đổi tín hiệu CAN logic $\leftrightarrow$ CANH/CANL.
- Kết nối STM32 với CAN Bus và node bên ngoài.
- Cho phép kiểm tra giao tiếp giữa ECU và Tester/ECU khác.

2.2. SOFTWARE ARCHITECTURE

07

Application

- Chứa logic điều khiển và mô phỏng ECU.
- Xử lý dữ liệu Engine, Diagnostic và giao diện người dùng.
- Không truy cập trực tiếp CAN hardware.

UDS

- Xử lý các request chẩn đoán từ tester.
- Phân tích SID, kiểm tra request và tạo response.
- Giao tiếp với DID Manager, DTC Manager và Application.

ISO-TP

- Đóng vai trò middleware vận chuyển dữ liệu giữa UDS và CAN.
- Phân mảnh bản tin dài thành nhiều CAN frame.
- Ghép các frame nhận được thành một UDS message hoàn chỉnh.

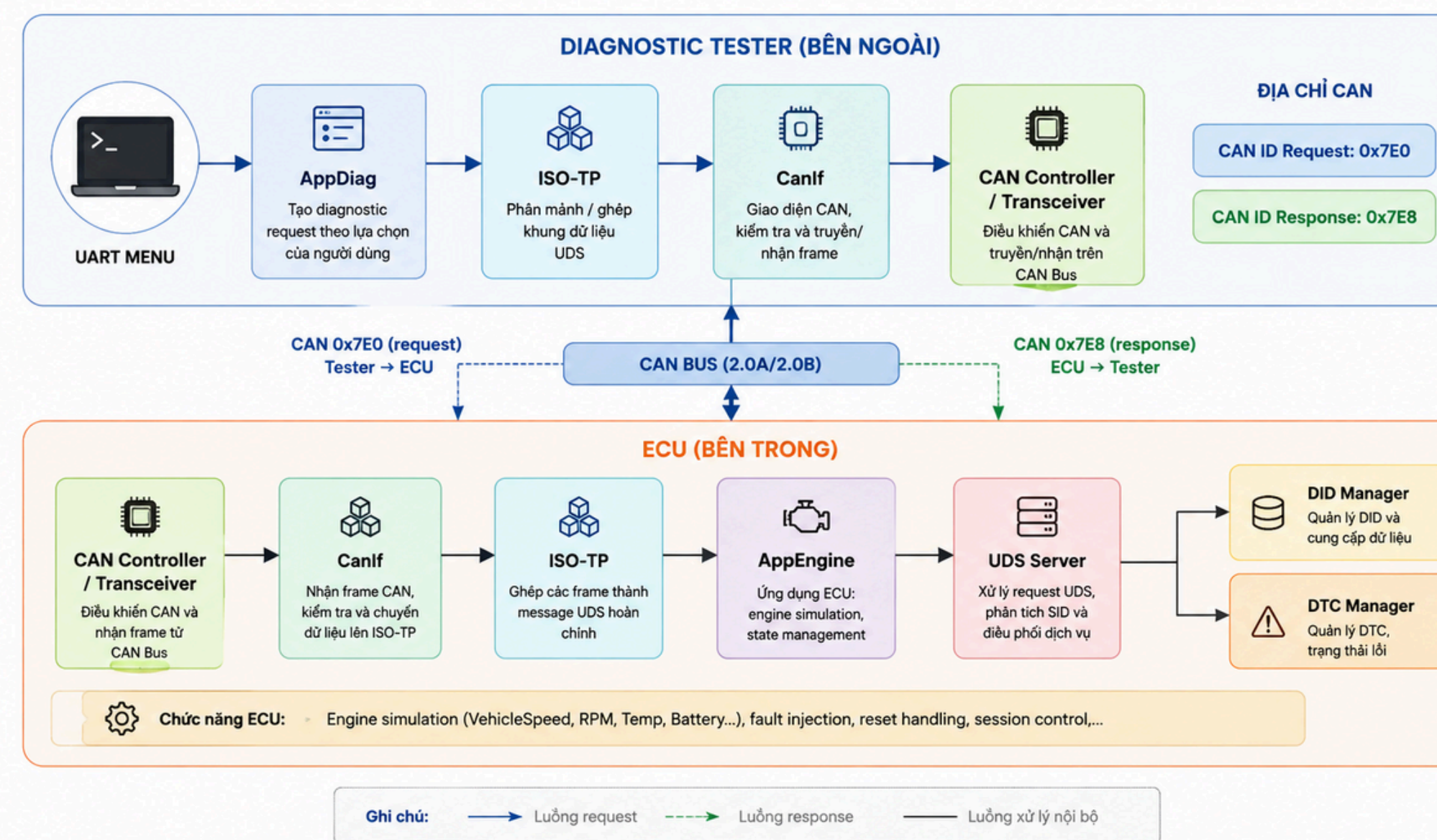
CAN_IF

- Cung cấp API truyền và nhận CAN frame cho các tầng phía trên.
- Kiểm tra CAN ID, DLC và dữ liệu trước khi truyền.
- Xử lý RX interrupt và quản lý hàng đợi frame.

CAN Driver

- Là lớp giao tiếp trực tiếp với CAN peripheral của STM32.
- Sử dụng STM32 HAL để truyền, nhận và cấu hình CAN.
- Che giấu chi tiết phần cứng đối với các tầng phía trên.

Sơ đồ luồng giao tiếp Diagnostic Tester – ECU



2.3. CẤU TRÚC PROJECT

08

Sơ đồ cấu trúc project

Applications/	
EC/main_EC.c	Entry point mẫu cho ECU
Diag/main_diag.c	Entry point mẫu cho tester
Core/	
Inc/	Header CubeMX: main, interrupt, HAL config
Src/	Entry point đang được build, ISR, MSP, system
Startup/	Vector table và startup Cortex-M3
Modules/	
app/	Ứng dụng ECU và tester
app_engine.[ch]	Engine simulation, DTC update, UDS server integration
app_diag.[ch]	Menu UART, UDS client và hiển thị response
did/	DID table và dữ liệu VIN/SW/live signal
dtc/	DTC table, trạng thái active, serialisation
drivers/	
canif/	CAN HAL wrapper, RX queue và callback FIFO1
isotp/	ISO 15765-2 SF/FF/CF/FC transport
uds/	UDS dispatcher và các service 0x10/11/19/22/3E
Drivers/	STM32 HAL và CMSIS do STM32Cube cung cấp
docs/	
design/	Architecture, module diagrams và detailed design
process/	Quy trình quản lý cấu hình
requirements/	SYS/SWR và traceability matrix
tests/	Unit, integration, system và acceptance test specs
STM32F103C8TX_FLASH.ld	Linker script
.cproject, .project	Cấu hình STM32CubeIDE/Eclipse

Applications

- Chứa các chương trình chính của ECU và tester.
- Là điểm bắt đầu của logic chẩn đoán và xử lý ứng dụng.

Modules/drivers

- Chứa CAN_IF và các interface kết nối giữa CAN Driver với tầng UDS.
- Đảm nhiệm giao tiếp TX/RX và quản lý dữ liệu CAN.

Modules/middleware

- Chứa lớp ISO-TP xử lý giao tiếp vận chuyển.
- Đảm nhiệm phân mảnh, truyền và ghép các CAN frame thành UDS payload.

3.1. CAN – GIAO TIẾP VÀ GIAO THỨC TRUYỀN DỮ LIỆU

09

CAN (Controller Area Network) là giao thức truyền thông dùng để các node ECU trao đổi dữ liệu trên cùng một bus mà không cần kết nối trực tiếp từng cặp.

CAN: Controller Area Network (CAN)

- Giao tiếp truyền thông phổ biến trong hệ thống ô tô.
- Nhiều ECU có thể cùng kết nối trên một CAN Bus.
- Dữ liệu được truyền dưới dạng CAN Frame.
- Trong project, STM32 sử dụng bxCAN + HAL CAN để giao tiếp với bus.

CAN Frame: Cấu trúc dữ liệu cơ bản

| CAN ID | DLC | DATA (0–8 byte) |

- CAN ID: xác định message.
- DLC: số byte dữ liệu.
- DATA: payload của message.
- Một CAN Data Frame trong project được xử lý với tối đa 8 byte payload, sau đó ISO-TP được sử dụng khi dữ liệu dài hơn.

Cơ chế truyền: CAN hoạt động theo kiểu message-based

- Mỗi message được xác định bởi CAN Identifier (ID).
- Các node cùng kết nối trên một bus.
- Node nhận sẽ kiểm tra ID để xác định message cần xử lý.
- Trong project, EngineStatus sử dụng CAN ID 0x100 và DLC 8.

Luồng truyền trong hệ thống

Application -> CAN_IF_Transmit() -> STM32 HAL CAN -> CAN Controller -> CAN Transceiver -> CAN BUS -> Tester / ECU khác

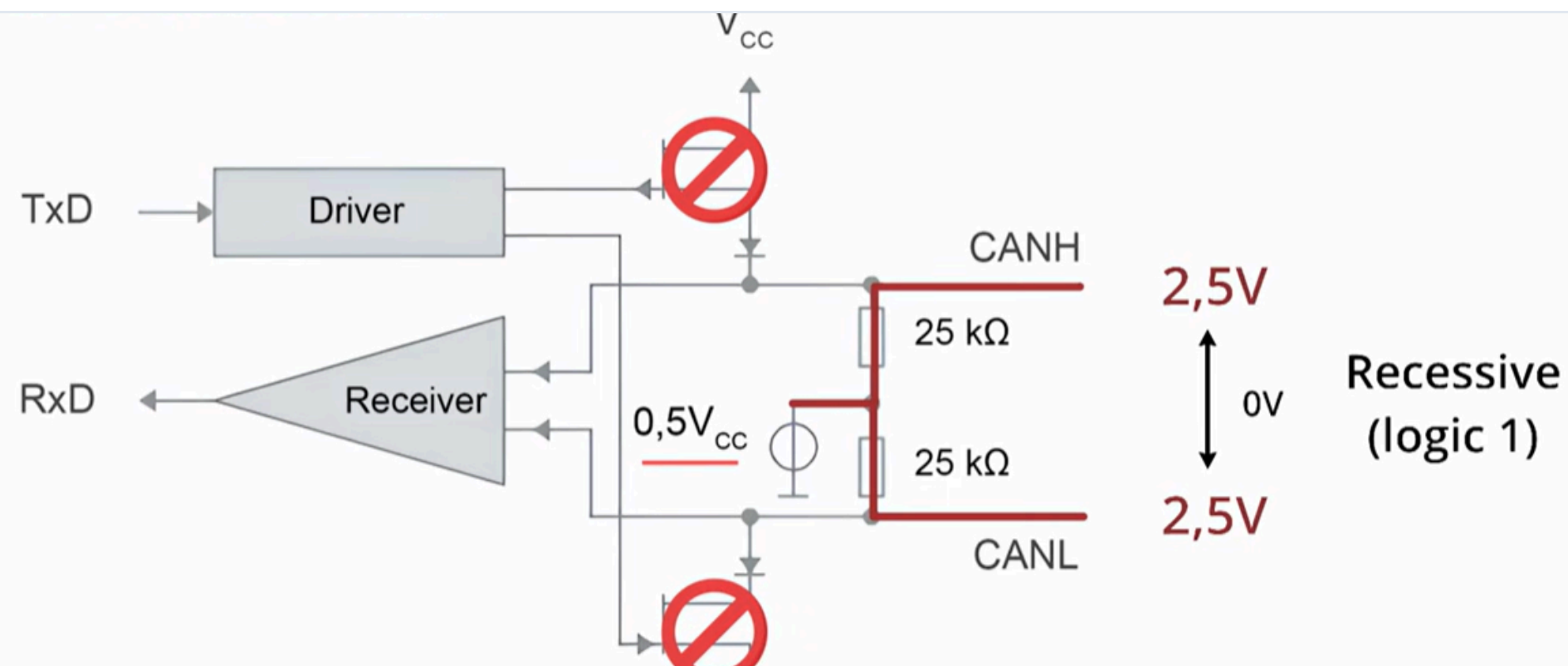
3.2. CAN Transceiver

09

CAN Transceiver (Bộ thu phát CAN) là thiết bị phần cứng đóng vai trò cầu nối trung gian giữa bộ điều khiển CAN (CAN Controller/MCU) và đường bus vật lý (CAN_H, CAN_L).

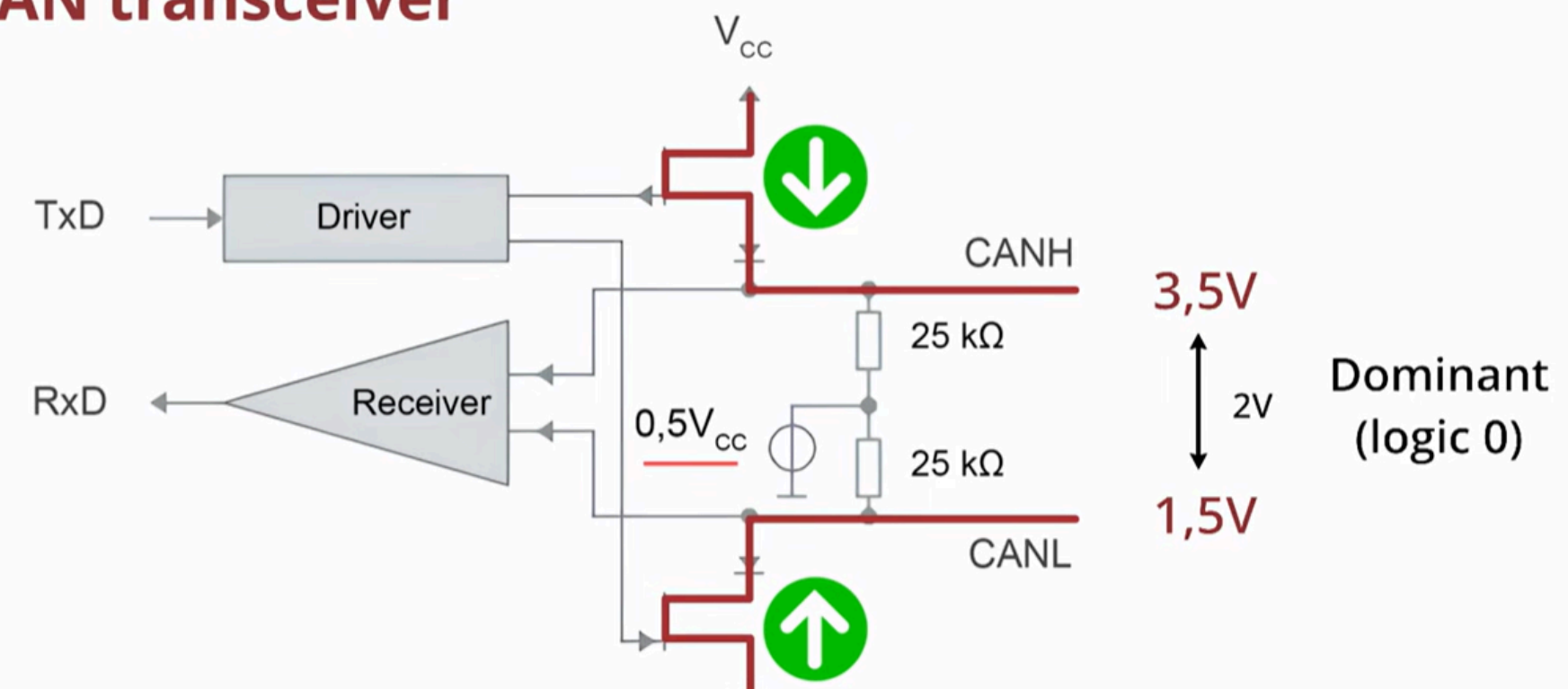
- Truyền dữ liệu (Transmitter): Chuyển đổi tín hiệu mức logic (TxD) từ micro-controller thành các mức điện áp vi sai tương ứng với trạng thái Dominant (logic 0) hoặc Recessive (logic 1) để đưa ra bus.
- Nhận dữ liệu (Receiver): Đọc chênh lệch điện áp vi sai trên đường bus CAN, chuyển đổi ngược lại thành tín hiệu logic chuẩn (RxD) gửi về vi điều khiển.

Mức 1



Mức 0

CAN transceiver



3.3. ISO-TP – 4 FRAME TYPES

SF — Single Frame

- Dùng cho message có payload nhỏ, tối đa 7 byte.
- Length của message được đặt trong byte đầu tiên.
- Không cần Flow Control.
- Ví dụ request VIN: 03 22 F1 90.

FF — First Frame

- Dùng để truyền phần dữ liệu còn lại sau First Frame.
- Mỗi frame có Sequence Number để kiểm tra thứ tự.
- Sequence Number tăng tuần tự và được sử dụng để phát hiện frame bị mất hoặc sai thứ tự.
- Ví dụ: 21 xx xx..., 22 xx xx....

CF — Consecutive Frame

- Dùng khi message có kích thước lớn hơn một CAN frame.
- Chứa tổng độ dài message và phần dữ liệu đầu tiên.
- Sau FF, sender chờ receiver gửi Flow Control.
- Ví dụ dạng: 10 xx

FC — Flow Control

- Được receiver gửi để điều khiển quá trình truyền multi-frame.
- Chứa các thông tin như Block Size và STmin.
- Giúp sender biết khi nào được tiếp tục gửi và tốc độ gửi phù hợp.
- Ví dụ dạng: 30 BS STmin.

Processing loop: IsoTp_Init() → IsoTp_Send()/IsoTp_OnCanFrame() → IsoTp_MainFunction()

3.4. UDS – DỊCH VỤ CHẨN ĐOÁN

UDS là tầng xử lý các yêu cầu chẩn đoán, nằm phía trên ISO-TP và CAN Interface.

0x10 — Diagnostic Session Control

- Dùng để chuyển ECU giữa các phiên chẩn đoán.
- Project hỗ trợ Default Session (0x01) và Extended Session (0x03).
- Session hiện tại được lưu trong context của UDS.
- Ví dụ: 10 03 → 50 03.

0x11 — ECU Reset

- Cho phép tester yêu cầu ECU thực hiện reset.
- Project hỗ trợ Soft Reset (0x03).
- ECU tạo positive response trước, sau đó mới xử lý trạng thái reset.
- Ví dụ: 11 03 → 51 03.

0x19 — Read DTC Information

- Dùng để đọc thông tin các mã lỗi được ECU quản lý.
- Request được chuyển tới DTC Manager để lấy dữ liệu.
- Project có các DTC demo như quá nhiệt động cơ và điện áp thấp.
- Hỗ trợ đọc DTC theo status mask.

3.5. UDS – DỊCH VỤ CHẨN ĐOÁN

13

UDS là tầng xử lý các yêu cầu chẩn đoán, nằm phía trên ISO-TP và CAN Interface.

0x22 — Read Data By Identifier

- Dùng để đọc dữ liệu theo DID.
- Request được chuyển tới DID Manager.
- Ví dụ:
 - F190 → VIN
 - F187 → Software Version
 - 0101–0104 → dữ liệu động của xe.

0x3E — Tester Present

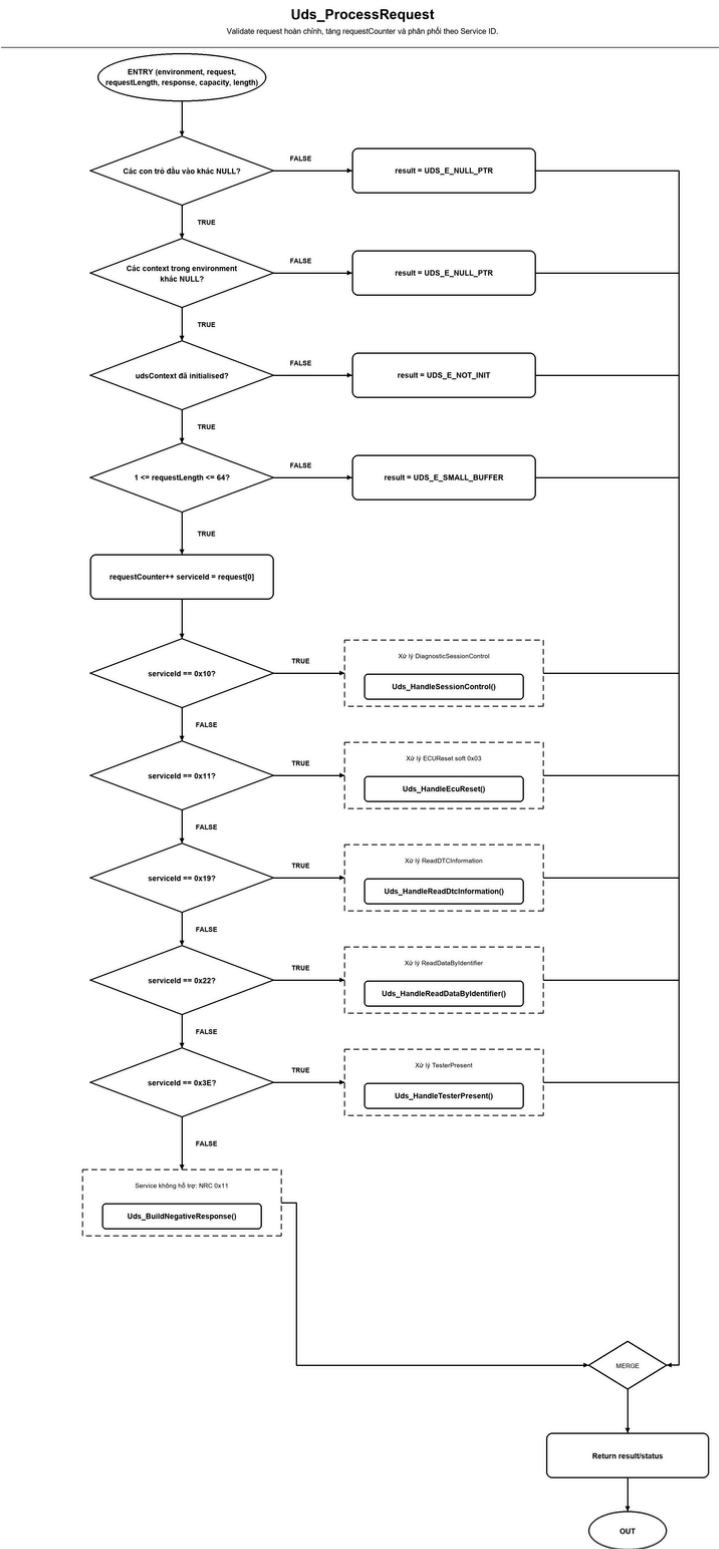
- Dùng để duy trì phiên chẩn đoán khi tester chưa gửi service khác.
- 3E 00 → ECU trả 7E 00.
- Hỗ trợ cả dạng phản hồi và dạng silent.

NRC – Negative Response

- Khi request không hợp lệ hoặc không thể thực hiện, ECU trả về dạng 7F <SID> <NRC>.
 - 0x11 – Service không được hỗ trợ
 - 0x12 – Sub-function không được hỗ trợ
 - 0x13 – Sai độ dài request
 - 0x22 – Điều kiện thực hiện không phù hợp
 - 0x31 – Request nằm ngoài phạm vi hỗ trợ

3.6. UDS – XỬ LÝ REQUEST / RESPONSE

Luồng xử lý UDS Request / Response



Dispatcher

Phân tích byte SID đầu tiên

- Byte đầu tiên của request được dùng để xác định service.
- UDS Dispatcher chuyển request tới đúng handler tương ứng.
- Trước khi xử lý, handler kiểm tra độ dài và tham số của request.

Positive response

Response thành công được tạo từ SID request

- SID positive = SID request + 0x40.
- Payload response chứa DID, dữ liệu hoặc thông tin diagnostic tương ứng.

Negative response

Request không hợp lệ sẽ trả về NRC

- Không hỗ trợ service → 7F 11 11.
- Sai độ dài → 7F SID 13.
- DID không tồn tại → 7F 22 31.
- Điều kiện chưa phù hợp → 7F SID 22.

4.1. APPLICATION – ENGINE STATUS

15

APP ENGINE

Mô phỏng tín hiệu xe/động cơ, cung cấp live data và báo điều kiện lỗi cho main.c

ĐẦU VÀO

Thời gian từ main.c
currentTimeMs
Mỗi 100 ms mới tăng một step
Module không tự đọc clock

Giá trị ép từ tester
Speed, RPM, temperature, pin
AppEngine_SetSignals()
dùng để chủ động tạo fault

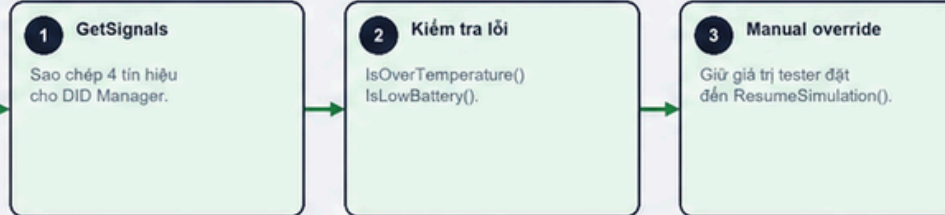
BÊN TRONG APP ENGINE

App Engine — Engine ECU

MÔ PHỎNG TỰ ĐỘNG



ĐỌC DỮ LIỆU VÀ FAULT



ĐẦU RA

Live signals
Vehicle speed
Engine RPM
Temperature và battery

Fault flags
overTemperatureActive
lowBatteryActive
trả về cho main.c

Trạng thái hàm
APP_E_OK
NULL_PTR / NOT_INIT
không chứa CAN/UDS

CÁC HÀM CÔNG KHAI

Init() / RunSimulationStep()
Khởi tạo và chạy ramp

GetSignals() / SetSignals()
Đọc hoặc ép toàn bộ tín hiệu

ResumeSimulation() / IsFault()
Chạy lại và kiểm tra lỗi

VÍ DỤ: TESTER ÉP NHIỆT ĐỘ 120°C



Engine data

4 thông số chính được quản lý

- Vehicle Speed – tốc độ xe.
- Engine RPM – tốc độ quay động cơ.
- Engine Temperature – nhiệt độ động cơ.
- Battery Voltage – điện áp nguồn.

Scheduler

- AppEngine_MainFunction(currentTimeMs) được gọi thường xuyên trong main loop.
- Module tự kiểm tra thời gian kể từ lần truyền trước.
- Khi đạt chu kỳ 1000 ms, module gửi một bản tin mới.
- Nếu gửi thành công, thời điểm truyền cuối cùng được cập nhật.

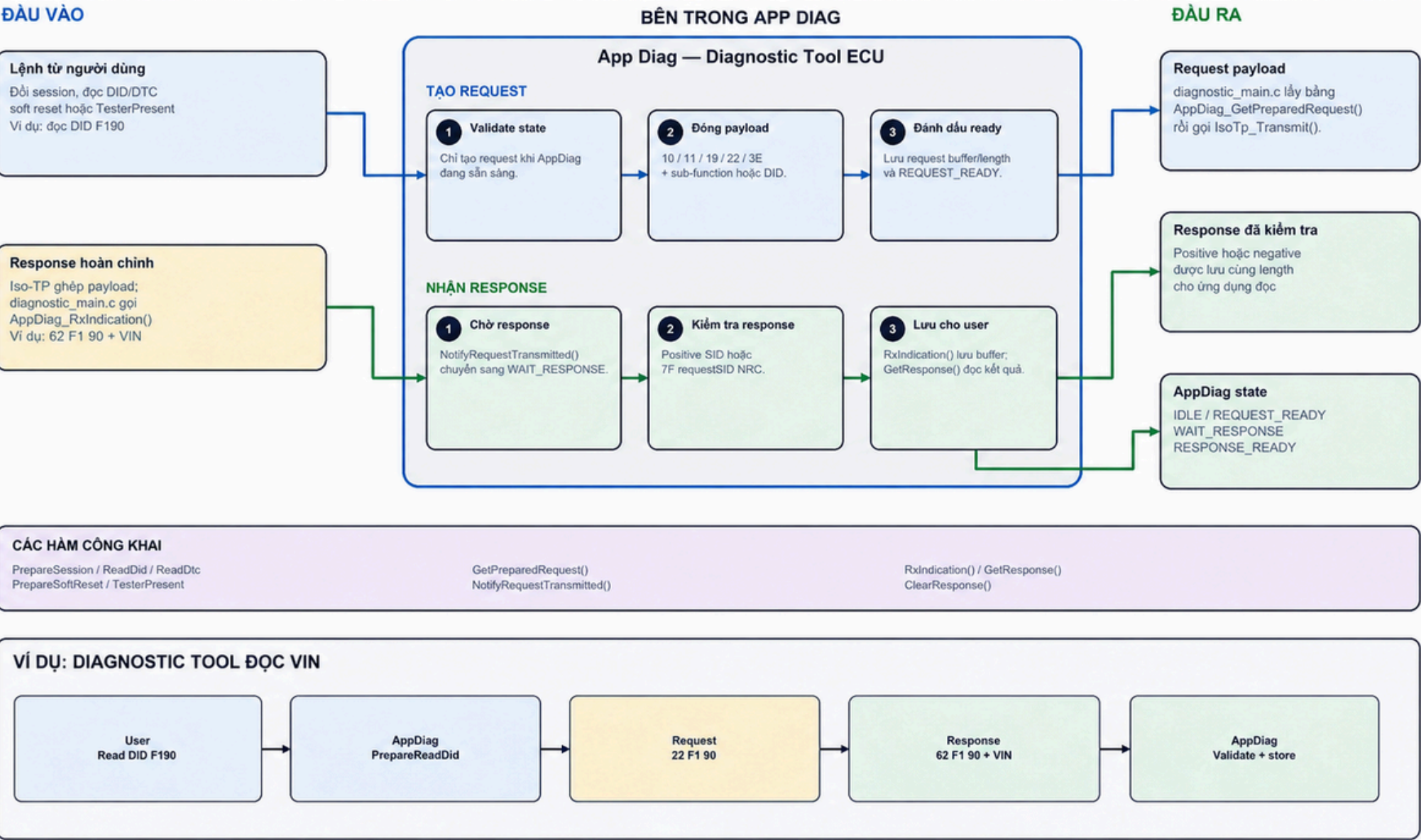
CAN payload

- Dữ liệu được encode theo Big Endian.
- CAN ID sử dụng: 0x100.
- DLC: 8 byte.

4.2. APPLICATION – DIAGNOSTIC FLOW

APP DIAG

Đại diện ứng dụng Diagnostic Tool: tạo request UDS, theo dõi request đang chờ và kiểm tra/lưu response



Đọc VIN

22 F1 90 → 62 F1 90 <VIN>

Đọc Software Version

22 F1 87 → 62 F1 87 <Version>

ECU Reset

11 03 → 51 03

Tester Present

3E 00 → 7E 00

Đọc DTC

19 02 <StatusMask> → 59 02 ...

Đọc dữ liệu động

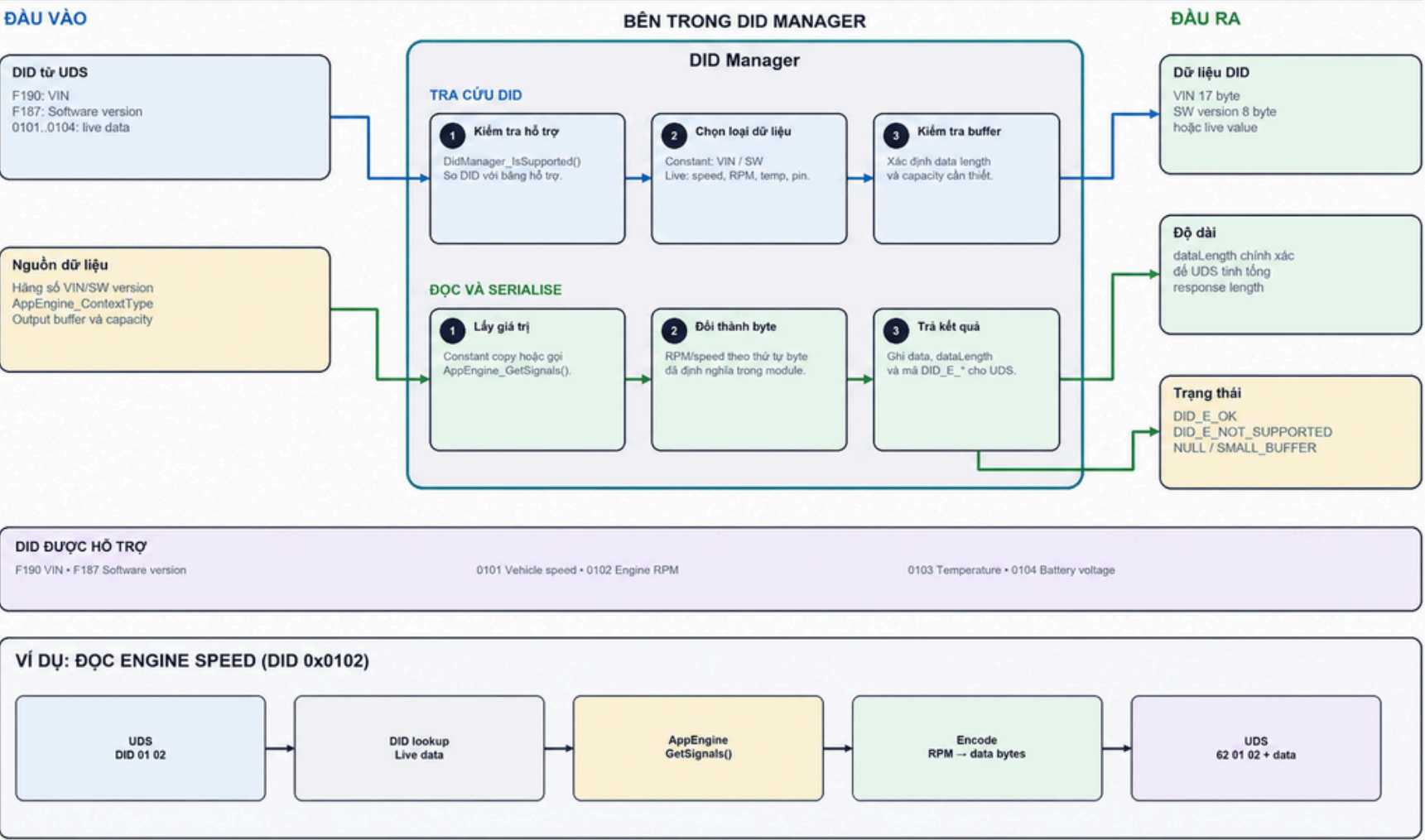
22 01 01 → VehicleSpeed
22 01 02 → EngineRPM

22 01 03 → EngineTemp
22 01 04 → BatteryVoltage

4.3. DID & DTC MANAGERS

DID MANAGER

Ảnh xạ Data Identifier sang đúng nguồn dữ liệu và đóng gói giá trị thành byte cho UDS

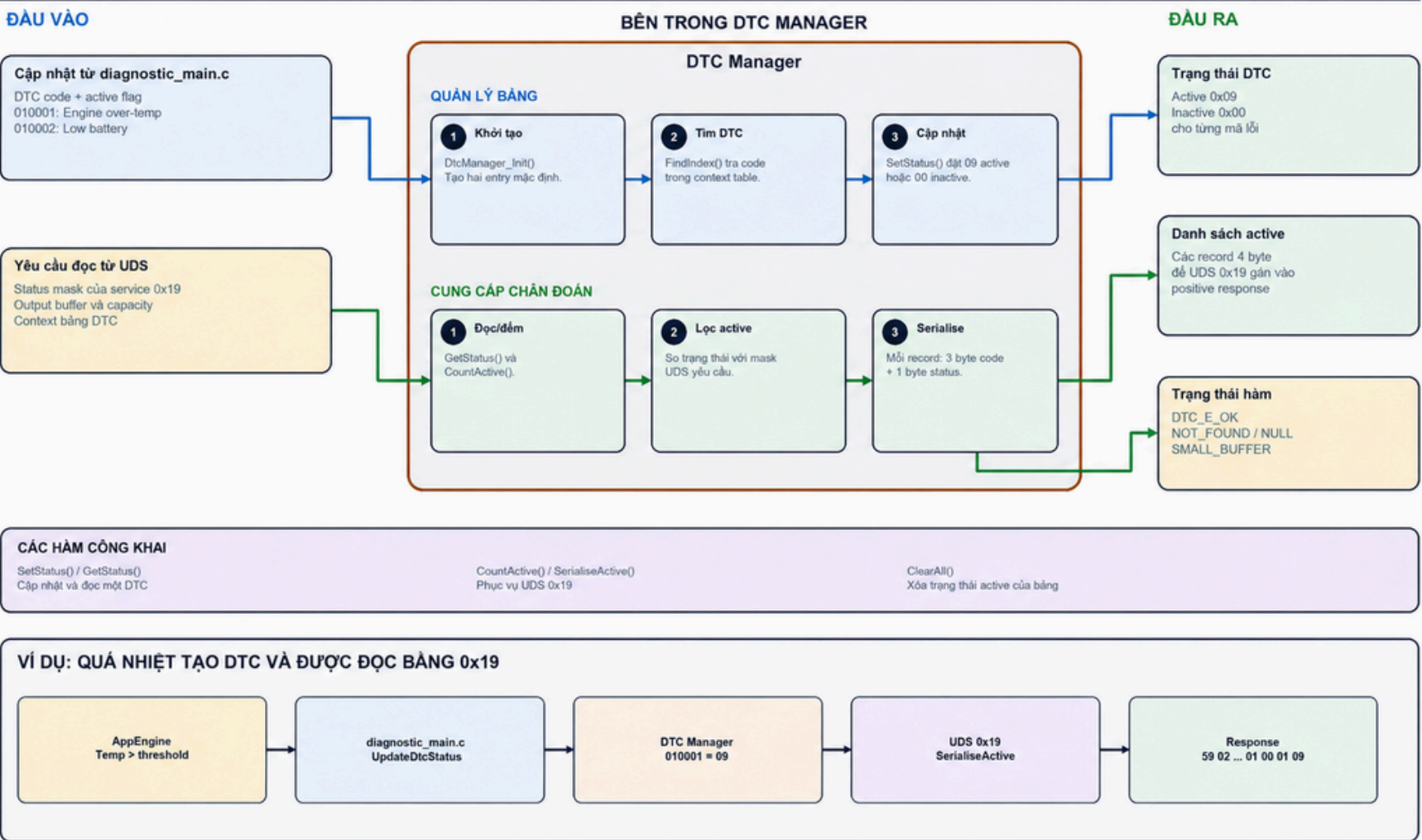


DID Manager – Quản lý dữ liệu định danh

- Quản lý các DID (Data Identifier) được ECU hỗ trợ.
- Lưu trữ thông tin và dữ liệu tương ứng với từng DID.
- Tiếp nhận yêu cầu đọc dữ liệu từ UDS.
- Kiểm tra DID và trả về dữ liệu nếu hợp lệ.

DTC MANAGER

Sở hữu bảng mã lỗi, lưu trạng thái active/inactive và serialise DTC cho service UDS 0x19



DTC Manager – Quản lý mã lỗi

- Quản lý danh sách DTC (Diagnostic Trouble Code).
- Lưu thông tin và trạng thái của từng mã lỗi.
- Cung cấp dữ liệu DTC cho các dịch vụ chẩn đoán.
- Hỗ trợ đọc và kiểm tra trạng thái lỗi của ECU.

5. VERIFICATION – 4 TẦNG KIỂM THỬ

18

Artifacts in docs/tests/ and traceability matrix

Unit – Kiểm thử đơn vị

- UT-CAN / UT-DID / UT-DTC / UT-UDS / UT-ISO-TP
- Kiểm tra từng module độc lập.
- Đảm bảo chức năng và xử lý dữ liệu đúng theo thiết kế.

Integration – Kiểm thử tích hợp

- IT-001 → IT-006
- Kiểm tra sự kết nối giữa các module.
- Đảm bảo dữ liệu truyền đúng qua các lớp CAN → ISO-TP → UDS → Application.

System – Kiểm thử hệ thống

- ST-001 → ST-011
- Kiểm tra toàn bộ ECU theo luồng từ đầu vào đến đầu ra.
- Xác nhận hệ thống hoạt động đúng trong các tình huống chẩn đoán thực tế.

Acceptance – Kiểm thử nghiệm thu

- AT-001 → AT-004
- Kiểm tra trên phần cứng và luồng chẩn đoán thực tế.
- Kết hợp demo, fault injection và bằng chứng kiểm thử trên GitHub.

6. LÀM VIỆC NHÓM & QUẢN LÝ CẤU HÌNH

19

Thanh – Kiến trúc hệ thống

- Thiết kế kiến trúc tổng thể.
- Tích hợp CAN ↔ UDS và định hướng luồng dữ liệu.

Cường– CAN / CAN Interface

- Phát triển CAN Driver và CAN Interface.
- Đảm bảo giao tiếp CAN ổn định giữa các module.

Giang – UDS / Diagnostic Services

- Xây dựng các dịch vụ chẩn đoán UDS.
- Quản lý request / response và xử lý dữ liệu chẩn đoán.

Dũng– Kiểm thử & xác minh

- Xây dựng test case và thực hiện kiểm thử.
- Thu thập log, CAN trace và các bằng chứng kiểm thử.

Issue

Requirement

Branch

Isolated chan

Commit

Traceable cha

PR

Review

Merge

Integration

Test

Evidence

Quy trình quản lý code

- Mỗi yêu cầu được quản lý thông qua Issue.
- Phát triển trên branch riêng để tránh ảnh hưởng main.
- Commit rõ ràng, có thể truy xuất lại thay đổi.
- Pull Request được review trước khi merge.
- Sau khi tích hợp phải build và test lại.

Release gate

Build Pass → Test Pass → Evidence → Review → Merge
→ Chỉ tích hợp vào main khi đã đáp ứng các điều kiện kiểm tra.

KẾT LUẬN & DEMO

01 EngineStatus

- Mô phỏng trạng thái động cơ thông qua tín hiệu hệ thống.
- Kiểm tra chu kỳ và tính ổn định của tín hiệu.
- Kết quả đạt yêu cầu: 0x100 / 1000 ms $\pm 10\%$

02 UDS VIN

- Tester gửi yêu cầu đọc VIN.
- Request được truyền qua CAN $\rightarrow$ ISO-TP $\rightarrow$ UDS.
- ECU xử lý DID và trả về VIN.
- Kết quả: 22 F1 90 $\rightarrow$ 62 F1 90 + VIN

03 Diagnostics

- Thực hiện các dịch vụ UDS tiêu biểu.
- Kiểm tra phản hồi dương và phản hồi âm.
- Ví dụ: 10 03 / 11 03 / 19 02 FF / 3E 00

04 Evidence

- CAN Trace ghi nhận quá trình giao tiếp.
- UART Log ghi nhận hoạt động bên trong ECU.
- Tester Log dùng để đối chiếu request / response.
- Các kết quả được lưu làm bằng chứng kiểm thử.